Task 4 – Write Correct Expressions

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task04)

1. The four expressions

Expressions used in the program

```
fat_steps = round((fat - STD_FAT) / STEP)          # whole 0.1 % steps from 3.5 (can be negative)
snf_steps = round((snf - STD_SNF) / STEP)          # whole 0.1 % steps from 8.5
rate      = round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
amount    = round(litres * rate, 2)
valid_fat = MIN_FAT <= fat <= MAX_FAT              # comparison chaining
```

Worked example: litres 8.5, fat 4.2, SNF 8.7. fat_steps = round(7.000000000000002) = 7, snf_steps = round(1.999999999999993) = 2, rate = 30.00 + 7×0.40 + 2×0.30 = 33.40, amount = 8.5 × 33.40 = 283.90.

2. Precedence and associativity

Rule	Example	Result and reason
** binds tighter than unary minus	-2 ** 2	-4, because it is -(2**2); use (-2) ** 2 to get 4
** is right-associative	2 ** 3 ** 2	512 = 2 ** (3 ** 2), not (2**3)**2 = 64
unary minus on the right side of **	2 ** -1	0.5; the sign belongs to the exponent
* / // % are equal and left-associative	10 / 2 * 5	25.0 (left to right), not 1.0
+ and - are left-associative	5 - 3 - 1	1, i.e. (5-3)-1
comparison binds tighter than not, not tighter than and, and tighter than or	True or False and False	True: and is done first, then or

In the step formula the brackets `(fat - STD_FAT) / STEP` are essential: without them, `fat - STD_FAT / STEP` would divide only the constant.

3. Interpreter evidence (surprising results)

Python 3.12 interactive mode

```
>>> (4.2 - 3.5) / 0.1
7.000000000000002
>>> (3.1 - 3.5) / 0.1
-3.999999999999999
>>> (4.1 - 3.5) / 0.1
5.9999999999999964
>>> round((4.1 - 3.5) / 0.1)
6
>>> int((4.1 - 3.5) / 0.1)
5
>>> import math
>>> -2 ** 2
-4
>>> (-2) ** 2
4
>>> 2 ** 3 ** 2
512
>>> 2 ** -1
0.5
>>> 5 - 3 - 1
1
>>> 10 / 2 * 5
25.0
>>> int(-3.5), round(-3.5), math.floor(-3.5)
(-3, -4, -4)
>>> fat = 4.2
```

```
>>> 2.0 <= fat <= 7.0
True
>>> 2.0 <= fat and fat <= 7.0
True
>>> not 2.0 <= fat <= 3.0
True
>>> True or False and False
True
```

4. Three (and more) unexpected results explained

Expression	Result	Explanation
(4.2 - 3.5) / 0.1	7.000000000000002	None of 4.2, 3.5 or 0.1 is stored exactly, so the quotient is a hair above 7 (the brief quotes ...001; the last digits depend on the exact numbers but the point is the same). round() gives the intended 7.
(4.1 - 3.5) / 0.1	5.999999999999964	**Worse:** slightly *below* 6. int() would give 5 and a wrong rate; round() gives 6.
-2 ** 2	-4	Precedence: ** before unary minus.
int(-3.5), round(-3.5), math.floor(-3.5)	-3, -4, -4	int() truncates toward zero; floor goes to the lower integer; round(-3.5) goes to the even neighbour (-4).
round(2.5)	2	Banker's rounding (shown in Task 2).

5. round(), int() and math.floor() for negative values

For positive numbers int() and floor() agree (both cut off the fraction). For negatives they differ: int(−3.9) = −3 (toward zero), math.floor(−3.9) = −4 (toward minus infinity), round(−3.9) = −4 (nearest). For a step count we want the *nearest* whole step, which is exactly round(), because the only error is the tiny representation error of about 10^{-15}.

6. Comparison chaining and Boolean operators

`2.0 <= fat <= 7.0` means `2.0 <= fat and fat <= 7.0` with fat evaluated once; both give True for 4.2 (session above). It is how the range rule of Task 1 is written. `not` is the highest of the Boolean operators, then and, then `or`. Note that `2.0 <= nan <= 7.0` is False, so a range test also rejects a "nan" reading.

Alternative Concept and Complexity Analysis

Alternative: compute the steps with int() (truncation) or math.floor() instead of round().

steps.py (excerpt)

```
import math

STD_FAT, STD_SNF, STEP = 3.5, 8.5, 0.1
BASE, FAT_V, SNF_V = 30.00, 0.40, 0.30

def rate(fat, snf, how):
    fs = how((fat - STD_FAT) / STEP)
    ss = how((snf - STD_SNF) / STEP)
    return round(BASE + fs * FAT_V + ss * SNF_V, 2), fs
```

Measured (steps.py; SNF fixed at 8.5)

```
method        fat             raw steps  steps  rate (snf 8.5)
round()       4.2     7.000000000000002      7           32.80
int()         4.2     7.000000000000002      7           32.80
math.floor    4.2     7.000000000000002      7           32.80
round()       3.1    -3.999999999999999     -4           28.40
int()         3.1    -3.999999999999999     -3           28.80
math.floor    3.1    -3.999999999999999     -4           28.40
round()       4.1     5.999999999999996      6           32.40
int()         4.1     5.999999999999996      5           32.00
math.floor    4.1     5.999999999999996      5           32.00
```

Fat	round()	int()	math.floor()	Comment
4.2	7 steps → Rs 32.80	7 → 32.80	7 → 32.80	All agree because the raw value is just above 7.
3.1	−4 → Rs 28.40 (correct)	−3 → Rs 28.80 (Rs 0.40 too high)	−4 → Rs 28.40	int() truncates toward zero, so the farmer is overpaid; floor is right *by luck* (raw −3.999…).
4.1 (extra)	6 → Rs 32.40 (correct)	5 → Rs 32.00	5 → Rs 32.00	Raw 5.99…: both int and floor lose a whole step (Rs 0.40/L).

Cost: each method is a single operation, O(1) time and O(1) space; there is no performance reason to prefer one. The choice is purely about correctness, and only round() is correct for both signs and for representation errors of either direction.

Evidence

Correct expressions with several demonstrated surprises, each explained. Source: Task04/steps.py and the interactive session.